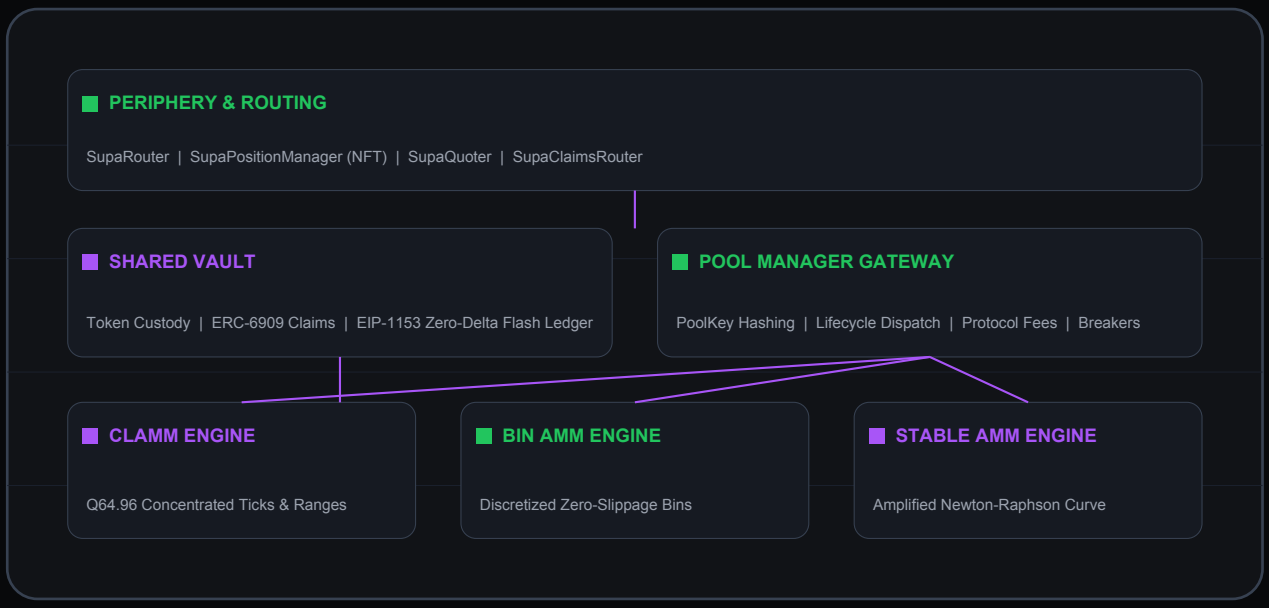
SUPADAO DEX / OFFICIAL PROTOCOL PUBLICATION

Developer Documentation

Contracts, integration paths, testing and deployment controls



VERSION 1.0.0 · 15 OCTOBER 2026 · UNDER TECHNICAL REVIEW

Institutional Notice: Repository-derived documentation based on audited smart contract implementations. No independent external audit or public deployment was confirmed.

Document Control & Verification Status

This technical publication is reproducible directly from verified Markdown specifications in the SupaDAO DEX repository. It describes inspected smart contract code and does not constitute legal, tax, or investment advice.

Contents & Architecture Sections

01 Environment
02 Contracts
03 Create and address a pool
04 Swap integration
05 Liquidity and positions
06 Events and errors
07 Deployment
08 Known integration constraints

Version: 1.0.0 | Date: 2026-10-15 | Status: Under Technical Review

Environment

Foundry configuration selects Solidity 0.8.26, Cancun EVM, via-IR compilation, 1,000,000 optimizer runs, 1,024 fuzz runs, and invariant runs/depth of 256/64.

```
forge build
forge test
forge snapshot
```

Contracts

Core contracts are `SupaVault`, `SupaPoolManager`, `PluginDispatcher` and the three curve engines (`CLAMMEngine`, `BinAMMEngine`, `StableAMMEngine`). Periphery contracts are router, quoter, position manager, claims router and fee collector. Interfaces, events, errors and user-defined value types live in separate directories under `contract/`.

Create and address a pool

Order token addresses strictly (`currency0 < currency1`), choose fee tier, spacing, plugin address and `CurveType`, then call manager initialization with a valid Q64.96 square-root price. Pool identity is `keccak256(abi.encode(PoolKey))`. Initialization can be restricted by a configured `PluginWhitelist`.

Swap integration

Use checked-in router structs and `PathKeyLib` encoding. Quote current state using `SupaQuoter`, specify a deadline and minimum output or maximum input, then approve the required ERC-20 or ERC-6909 claim allowance. The vault callback must finish with zero net deltas.

Liquidity and positions

The position manager tokenizes liquidity as ERC-721 positions. Mint, increase and decrease calls settle through the vault. Fees can be synchronized via `syncFees` and collected as ERC-6909 claims.

Events and errors

Consume declarations from `contract/events/` and `contract/errors/`. Never scrape revert strings: the implementation intentionally uses custom errors. Index `Initialize`, `ModifyLiquidity`, `Swap`, fee events and vault settlement events to reconstruct off-chain state.

Deployment

`DeploySupaDex.s.sol` orchestrates the contract suite and governance handoff; `VerifyDeployments.s.sol` checks deployment integrity. Validate constructor/proxy initialization, manager authorization, plugin whitelist, circuit breaker, fee controller, roles and timelock before broadcasting.

Known integration constraints

The stable engine assumes compatible token units, the bin engine remains in its active bin, the LVR plugin has a known tick update limitation, and no universal auto-router is bundled on-chain. Public addresses must come from a canonical signed manifest, not example configuration.